nanoGPT: architecture and hyperparameter experiments

Group 1 | block_size=64 | n_layer=4

Deep Learning Assignment - CSCE, University of North Texas

Team contribution: Group 1 here means the (64-character, four-layer) experiment partition. Use your existing term-project team and one representative for the combined submission.

Prepared from 32 completed training experiments. This report analyzes measured logs and generated text; it does not substitute expected trends for observed outcomes.

1. Introduction

nanoGPT is a compact implementation of a decoder-only transformer language model. It learns a conditional distribution over the next token using causal self-attention and a position-wise feed-forward network. In this study, tokens are individual characters from Tiny Shakespeare. Teacher-forced training predicts each following character from preceding characters, while generation repeatedly samples a prediction and appends it to the input.

The architecture combines learned token and position embeddings, repeated pre-normalization transformer blocks, residual connections, and a final vocabulary projection. Causality prevents a training position from reading its own future targets. The model can learn spelling, punctuation and local dialogue patterns within its context window, but those mechanisms alone do not establish reliable long-range narrative understanding.

Research question

For a fixed 64-character context and four transformer blocks, how do attention head count, embedding width, microbatch size, update budget and dropout change predictive loss, runtime, memory use and sample quality? A balanced five-factor design evaluates every combination of two values for each factor (2^5 = 32 runs).

Measured outcome	Observed value
Completed configurations	32 / 32, each trained from scratch
Lowest full-validation NLL	1.7599 (g1_15)
Character perplexity at that setting	5.812
Highest full-validation NLL	2.2971 (g1_02)
Sum of active run times	12.40 minutes

Pages 2-3 form the standalone code-reading contribution. Pages 4-10 describe the setup, actual results, visualizations, interpretation and reproducibility. When present, pages 11-12 add the controlled bonus study. Individual configurations, per-update logs, evaluation curves and samples accompany the report.

02 Code analysis: model.py

The analysis below refers to the pinned official source [1]. The experiment runner imports this model without changing its architecture.

1. Causal mask. The scaled-dot-product attention path passes `is_causal=True`. The fallback registers a lower-triangular buffer and replaces scores for future positions with negative infinity before softmax. Each query can attend to itself and earlier tokens only; the scaling is 1/sqrt(head dimension). Availability of the SDPA API does not guarantee a Flash Attention kernel is used on every GPU.

2. LayerNorm purpose and position. LayerNorm normalizes each token across its embedding features with learned gain and optional bias, which helps control activation scales during optimization. This is a pre-normalization architecture: ln_1 precedes attention, ln_2 precedes the MLP, and ln_f follows the entire stack. Its statistics do not depend on other batch examples.

3. One block forward pass. Compute u = x + Attention(LayerNorm(x)); then return u + MLP(LayerNorm(u)). Attention projects into queries, keys and values, splits the embedding into heads, mixes permitted positions, concatenates heads and projects back. The MLP expands C to 4C, applies GELU, projects to C and applies dropout. Residual additions preserve an identity path for information and gradients.

4. Positional embeddings. Token embeddings (B,T,C) are added to learned absolute position embeddings (T,C) indexed from 0 through T-1. Their table has block_size rows. Embedding dropout follows the sum. These are learned vectors, not sinusoidal encodings or rotary position embeddings.

5. n_embd, n_head and n_layer. n_embd is residual-stream width C; n_head splits C into heads of size C/n_head, so divisibility is required; n_layer is the number of sequential blocks. Increasing C increases the dominant matrix parameter count approximately quadratically; increasing depth increases it roughly linearly. With fixed C, changing the number of heads redistributes the same projection matrices and does not increase their parameter count.

6. Dropout. During training, dropout randomly zeros activations and rescales retained values by 1/(1-p). It acts on the initial embeddings, attention probabilities, attention output and MLP output. Calling eval() disables module dropout; the attention function also explicitly receives dropout_p=0 during evaluation. This regularization can reduce co-adaptation but can also slow fitting.

End-to-end tensor path

Stage	Shape or operation
Input IDs	(B, T), integer character IDs
Token + position vectors	(B, T, C)
Four pre-norm residual blocks	(B, T, C)
Final norm + tied output projection	(B, T, 65) for training
Categorical prediction	Next character at each position

The output projection shares weights with the token embedding table. This report counts each unique parameter once, including positional embeddings. The model's default printed count subtracts positional embeddings, so it differs slightly from the totals in the results table.

[1] Official source: github.com/karpathy/nanoGPT, model.py, commit 3adf61e154c3fe3fca428ad6bc3818b27a3b8291.

03 Code analysis: train.py and sample.py

Training loop [2]

1. Learning-rate schedule. get_lr uses linear warmup, learning_rate * (it+1)/(warmup_iters+1), then cosine decay from learning_rate to min_lr. After lr_decay_iters it holds min_lr. This study fixes warmup_iters=100 and lr_decay_iters=2000 for both training durations, isolating the effect of additional updates under a shared schedule.

2. Gradient accumulation. Each microbatch loss is divided by gradient_accumulation_steps before backward. Gradients accumulate across microsteps, then are unscaled, clipped and used for one optimizer update. zero_grad clears them afterward. DDP synchronizes on the final microstep. Effective global batch size is microbatch size times accumulation steps times world size. This study uses accumulation=1 and one GPU.

3. Splits and np.memmap. The Shakespeare preparation script, rather than train.py, splits the raw text in order: first 90% training, final 10% validation. train.py reads the corresponding uint16 binary files. np.memmap exposes a file-backed array whose pages are loaded on demand, avoiding an eager copy of the entire dataset. Random contiguous windows are sampled separately within each split; targets are shifted by one character.

4. Loss function. model.forward uses cross_entropy on flattened logits (B*T,V) and targets (B*T), with ignore_index=-1. This is the negative log-likelihood of the correct next character and matches categorical language modeling. It accepts logits directly and performs numerically stable log-softmax internally; smaller loss is better.

5. block_size and batch construction. A batch contains B input windows of T=block_size characters and equally sized one-step-shifted targets. Sampling leaves room for the final target. A larger T provides longer context but increases token work and the attention score matrix size, which scales as T squared in ordinary attention. Group 1 fixes T=64.

6. max_iters and lr_decay_iters. max_iters controls the update budget; lr_decay_iters controls the schedule horizon. They are distinct. The original loop increments after an update and stops when iter_num is greater than max_iters, yielding an extra update from a zero start. The supplied runner instead performs exactly 1000 or 2000 updates, evaluates that final state, and saves complete resume state.

Sampling [3]

1. Temperature. Divide logits by a positive temperature before softmax. A value below 1 sharpens the distribution; a value above 1 flattens it. Zero is invalid in this implementation. top_k=1 can implement greedy selection; a very small temperature is only an approximation.

2. Top-k. Keep logits at or above the kth-largest score and set the remainder to negative infinity before renormalizing. k is clamped to vocabulary size; ties can retain more than k values. Sampling uses torch.multinomial.

3. Context window. generate crops the conditioning input to its last block_size tokens before every forward pass. The returned sequence retains the full generated text. Learned position indices restart from zero inside the current window. This implementation recomputes the window and has no key/value cache.

4. Strategy choice. Lower temperature and smaller k favor frequent, predictable continuations; higher temperature and larger k increase diversity and can increase errors. Fixed prompts and seeds make comparisons interpretable. Loss evaluation is separate from sampling and does not depend on temperature.

5. Starting generation. sample.py encodes its start string (a newline by default), optionally reads a FILE: prompt, and uses metadata character mappings if available. Otherwise it falls back to GPT-2 tokenization. The custom runner explicitly uses the verified 65-character metadata, preventing that incompatible fallback for the supplied minimal checkpoint.

[2-3] Official source: github.com/karpathy/nanoGPT, train.py, sample.py and data/shakespeare_char/prepare.py, commit 3adf61e154c3fe3fca428ad6bc3818b27a3b8291.

04 Experimental setup

Dataset and baseline

Tiny Shakespeare contains 1,115,394 characters and 65 unique characters. Sorting the character set defines the integer vocabulary. The first 1,003,854 characters form the training split; the final 111,540 form validation. The splits are contiguous and disjoint. The dataset URL and SHA-256 digest are recorded in data/shakespeare_char/manifest.json. There is no independent test set, so the validation-selected result is not an unbiased test estimate.

The baseline is g1_01: 4 heads, embedding width 128, batch size 8, 1000 updates and dropout 0.1. All runs start from the same seed, 1337, but different tensor shapes and dropout probabilities can consume random draws differently. No run initializes from the supplied checkpoint.

Factor	Values	Held fixed or varied
Context / blocks	64 / 4	Fixed for Group 1
Attention heads	4, 8	Varied
Embedding width	128, 256	Varied
Microbatch size	8, 16	Varied; accumulation=1
Optimizer updates	1000, 2000	Varied; exact update counts
Dropout	0.1, 0.2	Varied
Optimizer	AdamW	lr=0.001; betas=(0.9,0.99)
Other optimization	Weight decay 0.1; clip 1.0	min_lr=0.0001; warmup=100
Schedule horizon	2000 updates	Fixed for both budgets

Measurement protocol

Every 250 updates, including zero and the final update, the runner evaluates 20 fixed batches per split, each with 16 windows. Dropout is disabled and training RNG state is not consumed by evaluation. Fixed evaluation windows reduce comparison noise but are not independent replications. Per-update training loss is logged separately with dropout enabled.

Final full-validation NLL scores 111,539 next-character targets exactly once in nonoverlapping blocks of up to 64 characters, with no context across block boundaries. It is a blockwise conditional likelihood rather than a sliding-window score with maximum context at every character. Perplexity is exp(NLL). The generalization gap uses sampled validation minus sampled training loss under the same protocol.

Hardware: NVIDIA T1000 8GB; PyTorch 2.6.0+cu124; Python 3.12.14; precision float32. Active time includes training, evaluation, checkpoint writes and generation after initialization, excluding user idle time and dependency installation. CUDA timing is synchronized. CUDA allocation measures this process's tensors and allocator reservation, not total system GPU memory. Cross-hardware timings should not be pooled without qualification.

05 Results: all 32 configurations

H = attention heads; E = embedding width; B = microbatch size. Train is final fixed-batch evaluation loss with dropout off. Val* is final full-validation NLL, the ranking metric. Sec is active run wall-clock time. All losses use natural logarithms.

Run	H	E	B	Steps	Drop	Train	Val*	Sec
g1_01	4	128	8	1000	0.1	2.236	2.240	12.6
g1_02	4	128	8	1000	0.2	2.299	2.297	13.6
g1_03	4	128	8	2000	0.1	2.058	2.080	24.4
g1_04	4	128	8	2000	0.2	2.152	2.160	23.7
g1_05	4	128	16	1000	0.1	2.083	2.109	13.6
g1_06	4	128	16	1000	0.2	2.175	2.178	13.2
g1_07	4	128	16	2000	0.1	1.903	1.971	26.6
g1_08	4	128	16	2000	0.2	2.023	2.056	20.6
g1_09	4	256	8	1000	0.1	1.987	2.043	16.8
g1_10	4	256	8	1000	0.2	2.082	2.125	16.8
g1_11	4	256	8	2000	0.1	1.770	1.877	30.7
g1_12	4	256	8	2000	0.2	1.865	1.949	30.4
g1_13	4	256	16	1000	0.1	1.817	1.919	24.4
g1_14	4	256	16	1000	0.2	1.906	1.998	24.5
g1_15	4	256	16	2000	0.1	1.621	1.760	47.2
g1_16	4	256	16	2000	0.2	1.716	1.849	45.1
g1_17	8	128	8	1000	0.1	2.227	2.236	10.7
g1_18	8	128	8	1000	0.2	2.296	2.295	10.1
g1_19	8	128	8	2000	0.1	2.061	2.085	18.1
g1_20	8	128	8	2000	0.2	2.150	2.156	18.9
g1_21	8	128	16	1000	0.1	2.062	2.092	10.9
g1_22	8	128	16	1000	0.2	2.154	2.170	12.2
g1_23	8	128	16	2000	0.1	1.888	1.960	20.9
g1_24	8	128	16	2000	0.2	2.005	2.040	20.4
g1_25	8	256	8	1000	0.1	1.996	2.057	16.6
g1_26	8	256	8	1000	0.2	2.084	2.122	17.0
g1_27	8	256	8	2000	0.1	1.765	1.879	31.3
g1_28	8	256	8	2000	0.2	1.870	1.945	30.8
g1_29	8	256	16	1000	0.1	1.826	1.925	24.9
g1_30	8	256	16	1000	0.2	1.907	1.983	24.3
g1_31	8	256	16	2000	0.1	1.624	1.764	46.2
g1_32	8	256	16	2000	0.2	1.704	1.830	46.2

The lowest observed full-validation loss is 1.7599 in g1_15; the highest is 2.2971 in g1_02. These are observed single-seed outcomes, not estimates of the best possible hyperparameters. Unrounded values and sampled validation loss are available in results/summary.csv.

06 Training and validation curves

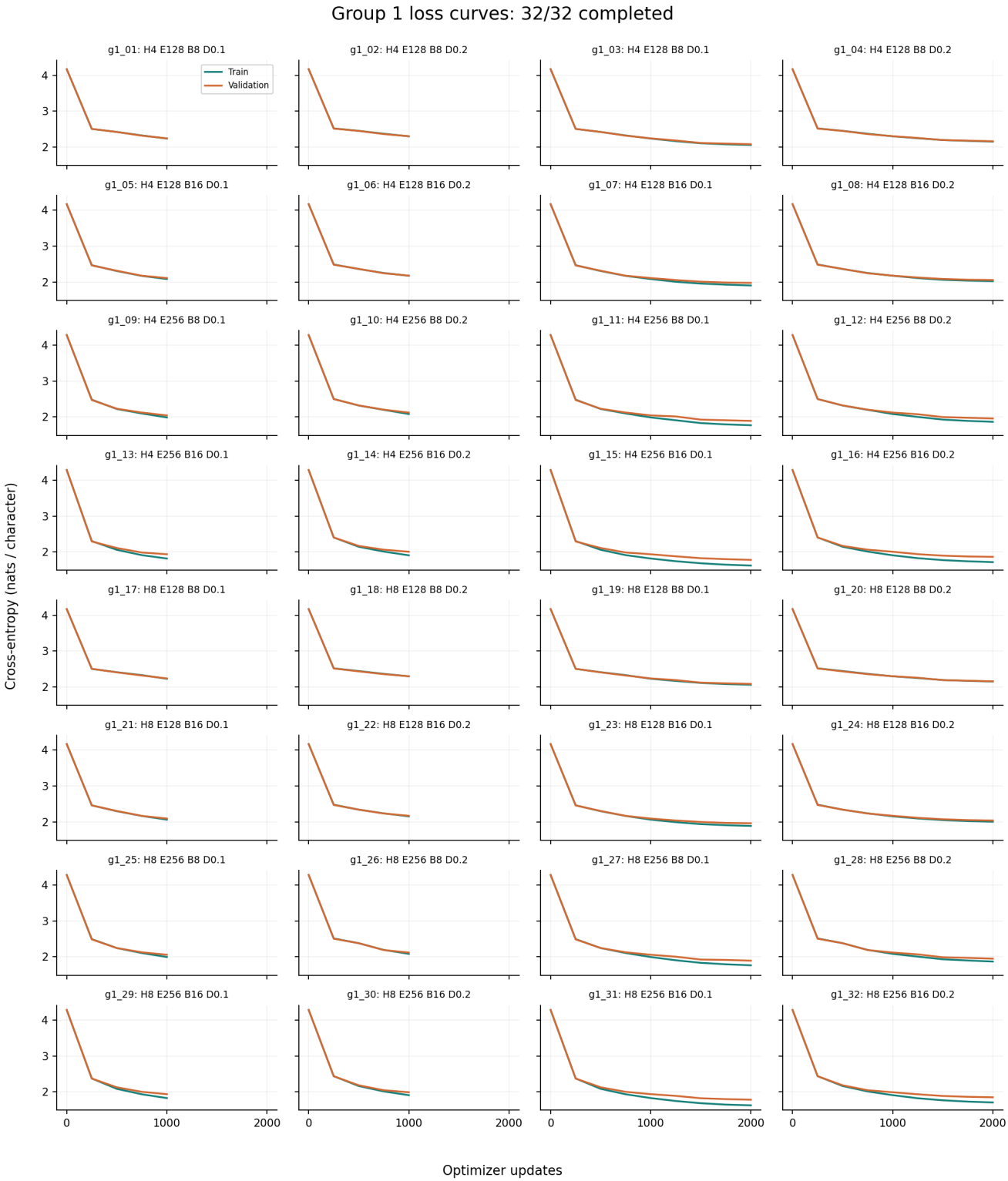


Figure 1. All 32 loss curves use the same axes. Green: fixed-batch train evaluation; orange: fixed-batch validation evaluation. Each panel identifies its head count, width, batch size and dropout. Curves stop at the configured 1000 or 2000 updates. Both durations use the same 2000-update learning-rate schedule, so the longer run receives additional lower-rate updates.

07 Effects and generalization gap



Figure 2. Final full-validation loss and the sampled train-validation gap. Negative gaps can arise because the held-out excerpt and sampled training excerpt differ in difficulty; they do not imply data leakage or prove an absence of overfitting.

Matched comparisons

For each factor, pair runs that agree on the other four factors. Delta means the higher factor value minus the lower value in final full-validation NLL. Negative deltas favor the higher value. This balanced average describes the observed grid; it is not a statistical significance test.

Factor change	Matched pairs	Mean NLL delta	Improved pairs
n_head: 4 to 8	16	-0.0045	11 / 16
n_embd: 128 to 256	16	-0.1935	16 / 16
batch_size: 8 to 16	16	-0.1214	16 / 16
max_iters: 1000 to 2000	16	-0.1519	16 / 16
dropout: 0.1 to 0.2	16	+0.0723	0 / 16

Changing batch size changes tokens per optimizer update. Changing max_iters changes the number of updates. Thus improvements from either factor may reflect increased data exposure, and this design does not hold total token or wall-clock budgets constant. Token counts are recorded explicitly: updates x batch_size x 64. The smallest budget is 512,000 characters and the largest is 2,048,000 sampled characters, with windows sampled with replacement.

Depth and context length are fixed in Group 1. This report therefore makes no experimental claim about whether six layers or a 128-character context would improve quality. Such comparisons require the corresponding teammates' measured results and a compatible evaluation protocol.

08 Generated text and qualitative checks

Each excerpt below is an actual 256-character continuation of the same newline + ROMEO: prompt. Generation uses seed 20263 and top_k=40. Text is unedited apart from display wrapping. The JSON files preserve the exact text and settings.

Lowest NLL: g1_15 | temperature 0.7

```
ROMEO:
I am they many will do I am as belikes as laster
To be you some as then stay tall haven from of the for atter
The streath fight of thee far of at I with to mings,
Too on thou art it be in throne stands the tear thou
to-makest threfore may more to maken th
```

Corpus-vocabulary word fraction: 86.2%; adjacent repeated word pairs: 0. Generated word forms absent from the training corpus include: atter, belikes, laster, maken, mings, streath, th, threfore. These are mechanical diagnostics, not grammar or coherence scores.

Highest NLL: g1_02 | temperature 0.7

```
ROMEO:
I a hatcod at foresore I yours belingiars lasguon.



WALECILARD:
Y pon.
G WARI RITI VO OLARDUS:
Tor seaner therele nol caris o o ith thane seant
I tuim the tonser oo o member d hit heave therin sthars thine.


SCLALIDIO:
Os keslant she lame thand aeme th
```

Corpus-vocabulary word fraction: 35.4%; adjacent repeated word pairs: 1. Generated word forms absent from the training corpus include: aeme, belingiars, caris, foresore, hatcod, heave, ith, keslant, lasguon, nol, olardus, oo. These are mechanical diagnostics, not grammar or coherence scores.

Same lowest-NLL model | temperature 1.0

```
ROMEO:
I am thy do have pair, I emont belikes as lasgue
To but gracess as wing.
Is that her be from of proice, and they hatheek himseove our enford
Worthe fuice, more servooloness or dis to Bolies rone stands,
So thine! and you hast thrant execlame throw arms th
```

Temperature changes the conditional sampling distribution without changing model weights or validation loss. Compare punctuation, valid word forms and local consistency across these paired samples. A short dialogue-style fragment can resemble the corpus while remaining grammatically or narratively weak. With a 64-character context, the model cannot directly condition on dialogue more than 64 characters back; these samples cannot demonstrate stable long-range dependencies.

09 Compute, capacity and interpretation

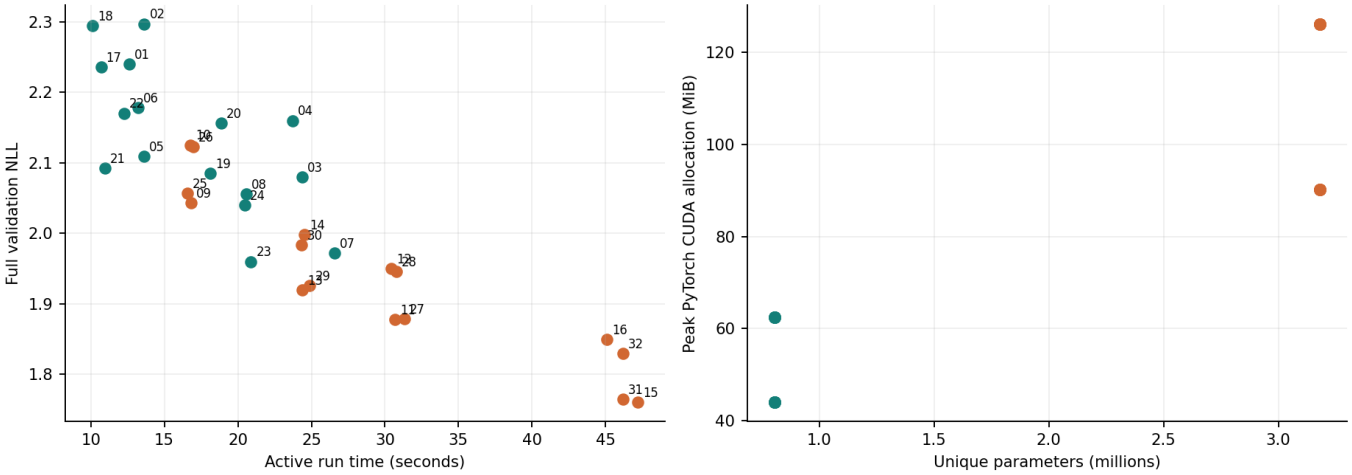


Figure 3. Measured active runtime versus final loss, and unique parameter count versus peak CUDA tensor allocation. Labels on the left identify run numbers. Green denotes embedding width 128; orange denotes width 256. Parameter storage is FP32; optimizer state, gradients and activations add to training memory.

Capacity and training budget

Changing embedding width from 128 to 256 gives a mean matched NLL delta of -0.1935 and a mean active-time ratio of 1.75. It improves 16 of 16 matched pairs. Changing the update budget from 1000 to 2000 gives a mean delta of -0.1519, improving 16 of 16 pairs. These values quantify the observed quality-cost tradeoff without assuming that larger models or longer training always win.

Head count changes the subdivision of a fixed embedding width rather than the size of the projection matrices. Any observed head-count effect therefore reflects the attention representation and optimization behavior, rather than an increase in the number of learned weights. Dropout changes training-time stochastic regularization while leaving the parameter count unchanged.

What these measurements establish

The balanced grid supports comparisons within this dataset, seed, context, depth and training schedule. It does not separate all causes of performance: batch size alters data exposure; optimizer dynamics change with width; validation is reused to choose the best result. Matched differences can also vary with the other factors, so a mean effect can hide interactions. See paired_effects.json for the full observed minimum and maximum paired deltas.

Only one training seed is used. There are no confidence intervals over independent training runs and no basis for significance claims. Fixed evaluation batches suppress measurement variation but do not remove training variance. The samples diagnose local text behavior, while loss measures next-character predictive accuracy; neither alone proves grammatical competence or long-range understanding.

10 Conclusion and reproducibility

Among the measured Group 1 settings, g1_15 achieves the lowest full-validation NLL (1.7599, perplexity 5.812). It uses 4 heads, width 256, batch size 16, 2000 updates and dropout 0.1. It is the best observed result in this single-seed grid, not a universally optimal configuration.

The observed mean head-count delta is -0.0045; the mean dropout delta is +0.0723. Together with width and training-budget comparisons, these measurements show why empirical evaluation matters: architectural intuition must be checked against both predictive performance and compute cost. Future work should repeat seeds, introduce a separate test split, compare equal-token or equal-time budgets, and combine teammates' context/depth experiments.

Supplied checkpoint reference

The instructor-supplied checkpoint loads strictly with the verified 65-character vocabulary. Its measured full-validation NLL is 1.8993 (perplexity 6.682). Its metadata specifies four layers, four heads, width 128, context 64 and dropout 0.0. It contains only model weights and model_args; the original seed, step count, optimizer state, split provenance and training time are unknown. It is not counted among the 32 new experiments, and no training history is reconstructed.

Reproduce and audit

Run the supplied Colab notebook from top to bottom with a GPU runtime. All 32 configurations are materialized before training. After interruption, rerun the training cell: completed runs are skipped and partial runs restore weights, optimizer, gradient scaler and RNG states from the last atomic checkpoint. Analysis verifies update counts, configuration matches, complete curves and sample lengths before the report can be built. GitHub instructions include the code, configurations, logs and figures; model binaries are excluded by default and can be uploaded as release assets.

References

[1] Karpathy, A. nanoGPT model.py. Pinned commit 3adf61e154c3fe3fca428ad6bc3818b27a3b8291. https://github.com/karpathy/nanoGPT/blob/3adf61e154c3fe3fca428ad6bc3818b27a3b8291/model.py

[2] Karpathy, A. nanoGPT train.py and data/shakespeare_char/prepare.py. Same pinned commit. https://github.com/karpathy/nanoGPT/tree/3adf61e154c3fe3fca428ad6bc3818b27a3b8291

[3] Karpathy, A. nanoGPT sample.py and GPT.generate. Same pinned commit. https://github.com/karpathy/nanoGPT/blob/3adf61e154c3fe3fca428ad6bc3818b27a3b8291/sample.py

[4] Karpathy, A. Let's build GPT: from scratch, in code, spelled out. Recommended course video. https://www.youtube.com/watch?v=kCc8FmEb1nY

[5] Vaswani et al. Attention Is All You Need (2017). https://arxiv.org/abs/1706.03762

[6] Mirzaeinia, A. Deep Learning Assignment: nanoGPT, supplied course PDF, 14 slides.

11 Bonus: context and position control

This bonus study uses the measured final checkpoint from g1_15 at update 2000. It changes the inference context without training a new model. All conditions predict exactly the same 4,096 held-out characters, so a difference cannot be explained by selecting easier target characters.

Question and a confound

Does access to more preceding text improve next-character prediction? A naive test shortens the input and resets its position indices to zero. Because nanoGPT learns absolute position embeddings, that operation changes both the text available to attention and the positions assigned to retained characters.

Available text	Ordinary reset policy	Aligned-position control
Last 8 characters	Position indices 0-7	Position indices 56-63
Last 16 characters	Position indices 0-15	Position indices 48-63
Last 32 characters	Position indices 0-31	Position indices 32-63
Last 64 characters	Position indices 0-63	Position indices 0-63

Protocol

The target indices are 4096 unique, evenly spaced positions from index 64 through the end of the validation split. Each input contains the L characters immediately preceding its target, for L in {8,16,32,64}. The target character itself is never an input. We hold the checkpoint, targets and precision fixed, disable dropout, and evaluate in batches of 32.

The aligned policy preserves the learned position indices of retained characters from the full 64-character window. It is an inference-time intervention and is not a separate model trained at a shorter context. At context 64 the two policies coincide. The custom positional path is checked against the official forward path and must produce bit-identical logits at that length.

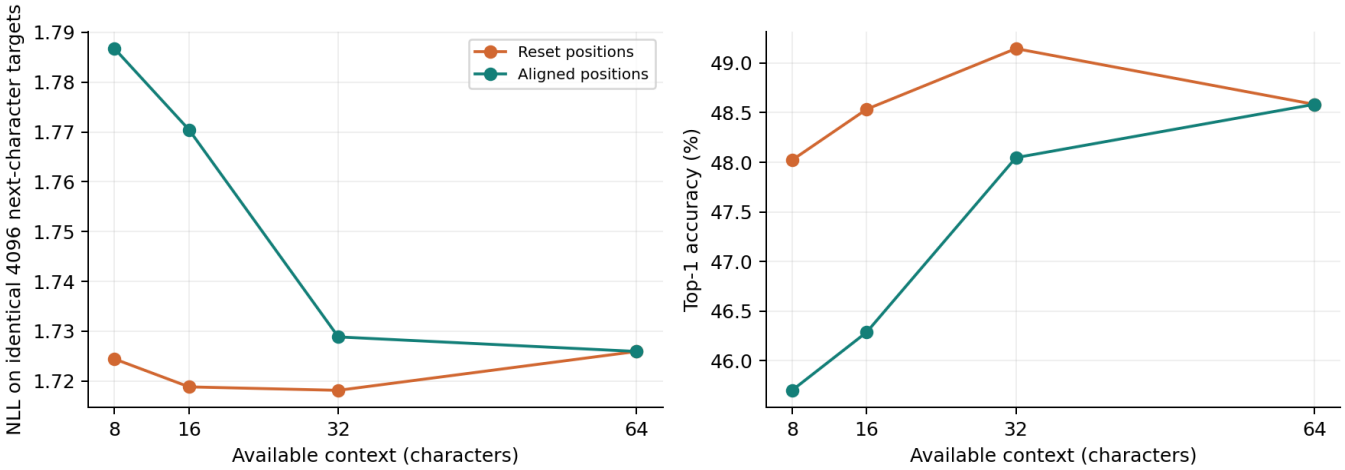


Figure 4. Actual paired-target NLL and top-1 accuracy under the two policies. The common 64-character point is evaluated once and reused; it is not a second independent measurement. Lower NLL and higher accuracy indicate better performance on this fixed target set.

12 Bonus: measured insights

Context	Policy	NLL	Accuracy	Penalty vs 64
8	Reset	1.7244	48.0%	-0.0015
16	Reset	1.7188	48.5%	-0.0071
32	Reset	1.7181	49.1%	-0.0078
64	Reset	1.7260	48.6%	+0.0000
8	Aligned	1.7867	45.7%	+0.0608
16	Aligned	1.7704	46.3%	+0.0445
32	Aligned	1.7289	48.0%	+0.0029
64	Aligned	1.7260	48.6%	+0.0000

With aligned positions, reducing context from 64 to 8 changes mean NLL by +0.0608 nats per character. Full context assigns higher probability to the correct target for 57.7% of anchors; the median paired penalty is +0.0165. Thus the mean effect should be interpreted alongside target-level variation, rather than assuming every prediction benefits equally.

At context 8, resetting rather than aligning position indices changes mean NLL by -0.0623. This quantifies positional sensitivity that a single naive context sweep would hide. The 64-character fixed-target baseline has NLL 1.7260; it differs from the main report's blockwise validation score because these anchors all receive a full preceding window.

Where does context help?

Target class	Targets	Aligned-8 penalty	Helped by 64
lowercase	2773	+0.0311	55.8%
punctuation	211	-0.4478	21.3%
uppercase	335	-0.1287	50.1%
whitespace	777	+0.3866	77.6%

The largest character-class penalty is for whitespace (+0.3866); the smallest is for punctuation (-0.4478). The lowest observed NLL occurs at context 32 with resets and 64 with aligned positions. These are descriptive observations about this checkpoint and target sample, not independent replications or evidence of statistical significance.

What the bonus adds

This study separates a context intervention from a learned-position confound, uses identical targets for paired comparisons, and examines which kinds of prediction change. It tests a concrete property of the trained model instead of adding an uncontrolled hyperparameter setting. Per-target losses, prediction entropy, environment details and the checkpoint hash are retained in results/bonus_context/.

The probe uses one validation-selected checkpoint and the same validation split, so it is descriptive rather than an unbiased test-set claim. Nearby anchors share context. No training-seed confidence interval or significance test is asserted. The maximum context is 64 characters, so these results concern local predictive context and do not establish long-range narrative understanding.

Team integration: this is the (64,4) experiment partition's contribution. The existing term-project team should combine its evidence into one 8-12 page report and nominate one representative to submit the shared deliverable. Teammate experiments have not been imported here.